

Dobot ROS2 Bridge

Isaac Sim Extension – Technische Dokumentation
Bidirektionaler Digital-Twin-Kopplungsrahmen für den
Dobot Magician in NVIDIA Isaac Sim 5.1

Tobias Küster

12. Juli 2026

Inhaltsverzeichnis

1	Übersicht und Zielsetzung	3
1.1	Zielsetzung	3
2	Systemumgebung und Softwarestapel	4
2.1	Hardware	4
2.2	NVIDIA-Treiber und CUDA	4
2.3	NVIDIA Isaac Sim	5
2.4	ROS2-Integration in Isaac Sim	5
2.5	Python-Pakete und Laufzeitabhängigkeiten	5
2.6	Systemanforderungen (Zusammenfassung)	6
3	Einrichtung und Installation	6
3.1	Projektdateien	6
3.2	Extension-Manifest (<code>extension.toml</code>)	6
3.3	Installations-Hilfsskript (<code>install_extension.py</code>)	7
3.4	Erstinstallation	8
4	Robotermodell	8
4.1	URDF-Import	8
4.2	Gelenkstruktur des Dobot Magician	9
5	Systemarchitektur	10
5.1	Schichtenmodell	10
5.2	Hauptklassen	10
5.3	Datenfluss	10
5.3.1	Richtung 1: Echter Roboter → Simulation (Digital Twin)	10
5.3.2	Richtung 2: Simulation / UI → Echter Roboter	10
6	Kinematisches Modell	11
6.1	Koordinatenkonventionen (pydobot)	11
6.2	Isaac-Sim-Gelenkstruktur	11
6.3	Einheitenstrategie	12
6.4	Berechnungsformeln	12
6.4.1	joint_1 – Gierwinkel mit festem Offset	12
6.4.2	joint_2 – Direkte Übernahme	12

6.4.3	joint_5 – Parallelogramm-Kompensation	12
6.4.4	joint_6 – Endeffektor-Vertikalisierung	12
6.5	Vollständige Kinematik in <code>update_step()</code>	13
7	Implementierung	14
7.1	Lazy-Import-System	14
7.2	Klasse <code>DobotROSNode</code>	14
7.2.1	Initialisierung (<code>__init__</code>)	14
7.2.2	Methoden-Referenz	15
7.2.3	Hardware: Pneumatischer Sauggreifer	15
7.2.4	Fehlerbehandlung und Robustheit	16
7.3	Dynamic-Control-Kopplung	16
7.4	Klasse <code>DobotBridgeExtension</code>	17
7.4.1	Lebenszyklusmethoden	17
7.4.2	Frame-Update-Ablauf (<code>_on_update()</code>)	18
7.4.3	Ressourcenfreigabe (<code>_cleanup()</code>)	18
7.5	Threading-Modell	18
8	Benutzeroberfläche	19
8.1	Tab 0: Steuerung	19
8.2	Tab 1: Pick & Place	20
8.3	Status-Dashboard (immer sichtbar)	20
9	Funktionen	20
9.1	Pick-&-Place-Szenario (Teaching)	20
9.1.1	Datenstruktur der Wegpunkte	20
9.1.2	Aufzeichnung	20
9.1.3	Wiedergabe	21
9.2	Ziel-Würfel-Tracking	21
9.2.1	Würfel erstellen (<code>_spawn_target_cube()</code>)	21
9.2.2	Tracking-Schleife	22
9.3	Betrieb Schritt für Schritt	22
10	ROS2-Integration	23
10.1	Node-Konfiguration	23
10.2	Nachrichtenstruktur	23
10.3	ROS2-Kontext-Verwaltung	23
11	Konstanten (<code>constants.py</code>)	23
11.1	Farbwerte (ARGB-Hex)	24
11.2	Nullposition und Servo	24
11.3	USD-Pfade	24
12	Bekannte Einschränkungen	24

1 Übersicht und Zielsetzung

Die Extension `dobot_ros` implementiert eine **bidirektionale Echtzeit-Kopplung** zwischen einem realen **Dobot Magician**-Roboterarm und einem **NVIDIA Isaac Sim 5.1**-Simulationsmodell. Sie stellt damit einen vollständigen Digital-Twin-Kopplungsrahmen dar, der es ermöglicht, den physischen Roboter in einer virtuellen Simulationsumgebung zu spiegeln und umgekehrt Steuerbefehle aus der Simulation an die reale Hardware zu senden.

1.1 Zielsetzung

- **ROS2-Integration:** Die Extension startet intern einen ROS2-Node in Isaac Sim und publiziert Gelenkzustände auf dem Topic `/joint_states` (Typ: `sensor_msgs/msg/JointState`). Diese Schicht bildet die Grundlage für alle weiteren Ziele – ohne sie ist weder Synchronisation noch Steuerung möglich.
- **Digital Twin (Richtung 1 – Real → Sim):** Der echte Roboter bewegt sich, und das Simulationsmodell folgt in Echtzeit. Gelenkwinkel werden per USB über die `pydobot`-Bibliothek gelesen, kinematisch transformiert und über die Isaac-Sim-Dynamic-Control-API auf die Simulationsgelenkstruktur übertragen.
- **Fernsteuerung (Richtung 2 – Sim → Real):** Über das UI-Panel werden Jog-Befehle, Vertikalbewegungen, Vakuumgreifer-Schaltung, RC-Servo-Ansteuerung sowie automatisierte Pick-&-Place-Sequenzen an den echten Roboter gesendet.
- **Pick-&-Place (Teaching):** Bewegungsabläufe werden durch manuelles Verfahren des Roboters aufgezeichnet und anschließend vollautomatisch mit Sicherheitslogik wiedergegeben.
- **Würfel-Tracking:** Ein interaktiv in der Szene verschiebbarer Ziel-Würfel dient als Bewegungssollwert für den Roboterarm. Das Tracking funktioniert auch ohne verbundenen COM-Port.
- **Laborszene in Isaac Sim:** Die im Labor aufgebaute Szene – Dobot Magician mit Sauggreifer, Schutzkappe und DeLorean-Modellfahrzeug – wird geometrisch und kinematisch in NVIDIA Isaac Sim nachgebildet, sodass realer Aufbau und Simulation direkt vergleichbar sind.



(a) Realer Aufbau im LFP-Labor der HsH: Dobot Magician mit Sauggreifer, Schutzkappe (weißer Würfel) und DeLorean-Modellfahrzeug.



(b) Entsprechender Digitaler Zwilling in NVIDIA Isaac Sim: das USD-Modell spiegelt die reale Szene geometrisch und kinematisch.

Abbildung 1: Pick-and-Place-Szenario – real (links) und simuliert (rechts). Die bidirektionale Kopplung zwischen beiden Welten ist das zentrale Ziel dieser Extension.

Die folgenden Abschnitte beschreiben zunächst die Systemumgebung (Abschnitt 2), dann Einrichtung und Installation (Abschnitt 3), das Robotermodell (Abschnitt 4), die Gesamtarchitektur (Abschnitt 5), das kinematische Modell (Abschnitt 6) und schließlich alle Implementierungsdetails (Abschnitt 7).

2 Systemumgebung und Softwarestapel

Dieser Abschnitt dokumentiert die Hardware- und Softwareumgebung, in der die Extension entwickelt und betrieben wird.

2.1 Hardware

Komponente	Spezifikation
CPU	Intel Core i7-13620H (13. Generation, Hybrid-Architektur)
RAM	32 GB DDR5
GPU	NVIDIA GeForce RTX 4070 Laptop GPU, 8 GB VRAM (WDDM)
Betriebssystem	Windows 11 Home, Build 26200
Roboter	Dobot Magician, USB-Verbindung über COM3

2.2 NVIDIA-Treiber und CUDA

Software	Version
NVIDIA GeForce Treiber (Game Ready)	592.27 (2. März 2026)
CUDA Version (maximale Unterstützung durch Treiber)	13.1
CUDA Toolkit (installiert)	12.1.66
nvcc (CUDA Compiler Driver)	12.1, Build 32415258
CUDA-Pfad	C:\Program Files\NVIDIA GPU Computing Tool
Nsight Compute	2023.1.0
Nsight Systems	2022.4.2 / 2023.1.2
NVIDIA Nsight Visual Studio Edition	25.4.0.25287

Treiber-Mindestanforderung von Isaac Sim 5.1: Laut der mitgelieferten Datei `kit/driver-requirements.txt` werden NVIDIA-Treiber unterhalb von Version **537.58** (Windows) vom RTX-Renderer blockiert. Der installierte Treiber 592.27 liegt deutlich darüber und ist vollständig kompatibel.

2.3 NVIDIA Isaac Sim

Parameter	Wert
Isaac Sim Version	5.1.0-rc.19+release.26219.9c81211b.gl
Installationspfad	C:\NVIDIA_Isaac_Sim
Interne Python-Version	3.11.13 (gebündeltes Kit-Python)
Python-Executable	C:\NVIDIA_Isaac_Sim\python.bat
Kit-Executable	C:\NVIDIA_Isaac_Sim\kit\kit.exe

Isaac Sim bringt eine **eigene, isolierte Python-3.11.13-Laufzeitumgebung** mit. Diese ist vollständig von einer etwaigen System-Python-Installation getrennt. Alle Extension-Skripte laufen in dieser gebündelten Python-Umgebung.

2.4 ROS2-Integration in Isaac Sim

ROS2 wird nicht separat installiert. Isaac Sim 5.1 enthält eine eingebettete ROS2-Bridge-Extension unter:

C:\NVIDIA_Isaac_Sim\exts\isaacsim.ros2.bridge

Diese Bridge stellt ROS2 Humble **und** ROS2 Jazzy als vorkompilierte Shared Libraries bereit:

Parameter	Wert
Unterstützte ROS2-Distributionen	Humble (Standard) / Jazzy
RMW-Implementierung (Middleware)	rmw_fastrtps_cpp (Fast DDS)
Standard-Distro-Umgebungsvariable	ROS_DISTRO=humble
Library-Pfad (Humble)	isaacsim.ros2.bridge\humble\lib
rclpy-Pfad	isaacsim.ros2.bridge\rclpy

Die Umgebungsvariablen werden beim Start über `setup_ros_env.bat` gesetzt.

2.5 Python-Pakete und Laufzeitabhängigkeiten

Die Extension benötigt **keine** separate lokale Python-Umgebung. Alle Laufzeitabhängigkeiten laufen ausschließlich innerhalb der eingebetteten Python-Umgebung von Isaac Sim (`python.bat`).

Paket	Version	Bereitstellung
pydobot	≥ 1.3	via <code>omni.kit.pipapi</code> (Install-Button)
pyserial	≥ 3.5	via <code>omni.kit.pipapi</code> (Abhängigkeit)
rclpy	Humble-Build	eingebettet in Isaac-Sim-Bridge
sensor_msgs	Humble-Build	eingebettet in Isaac-Sim-Bridge
omni.ui	Kit-intern	bereits in Isaac Sim vorhanden
omni.ext	Kit-intern	bereits in Isaac Sim vorhanden
omni.isaac.dynamic_control	Kit-intern	bereits in Isaac Sim vorhanden
omni.kit.pipapi	Kit-intern	bereits in Isaac Sim vorhanden

2.6 Systemanforderungen (Zusammenfassung)

Komponente	Mindestanforderung / empfohlene Version
NVIDIA Isaac Sim	5.1.0 oder neuer
Isaac Sim Python	3.11 (intern, kein manueller Eingriff)
NVIDIA Treiber (Windows)	≥ 537.58 , empfohlen: 592.27+
CUDA Toolkit	12.1 oder neuer (für GPU-Physik)
GPU	NVIDIA RTX-fähige GPU, mind. 8 GB VRAM
RAM	mind. 16 GB (empfohlen 32 GB)
Betriebssystem	Windows 11 (64-Bit)
ROS2	Humble oder Jazzy (embedded, kein Standalone-Install)
Dobot Magician	USB-Verbindung, bekannter COM-Port

3 Einrichtung und Installation

Dieser Abschnitt beschreibt den Aufbau des Projekts, erklärt wie Isaac Sim die Extension lädt, und zeigt die nötigen Schritte zur Erstinstallation.

3.1 Projektdateien und Ordnerstruktur

Die Extension ist in drei Ebenen gegliedert: Der **Stammordner** enthält ausschließlich die von Isaac Sim direkt geladenen Dateien. Alles, was nicht zum Laufzeitbetrieb gehört, ist in Unterordnern ausgelagert.

Pfad	Beschreibung
Stammordner <code>dobot_ros/</code>	
<code>extension.py</code>	Hauptlogik: ROS-Node, Kinematik, DC-Kopplung, UI, Pick-&-Place, Würfel-Tracking
<code>constants.py</code>	Zentrale Farbwerte, USD-Pfad, Nullposition, Servo-Konfiguration
<code>__init__.py</code>	Exportiert <code>DobotBridgeExtension</code> als Extension-Einstiegspunkt
<code>extension.toml</code>	Isaac-Sim-Manifest: Titel, Version, Python-Modul
<code>Dobot Digital Twin.usd</code>	Isaac-Sim-Szene mit Dobot, DeLorean und Labor-tisch
Unterordner <code>Dokumentation/</code>	
<code>documentation.tex/.pdf</code>	Diese Dokumentation (Quelle und Ausgabe)
<code>build_docs.py/.bat</code>	Erzeugt das PDF aus der LaTeX-Quelle
<code>bilder/</code>	Abbildungen für die Dokumentation
Unterordner <code>Hilfsskripte/</code>	
<code>install_extension.py</code>	Kopiert Extension ins Omniverse-Roaming-Verzeichnis
<code>set_extension_path.ps1</code>	Setzt <code>OMNI_KIT_EXTENSION_PATH</code> per PowerShell
<code>calibrate_j1_offset.py</code>	Kalibriert den J1-Gierwinkel-Offset (Isaac-Sim-Script-Editor)
<code>load_dobot_via_URDF.urdf</code>	URDF-Beschreibung des Dobot Magician für den Isaac-Sim-URDF-Importer
<code>load_dobot_via_URDF/</code>	Fertig importierte USD-Szene des Roboters
<code>meshes/</code>	STL- und DAE-Meshes der Roboterglieder (von URDF referenziert)

Isaac Sim lädt beim Aktivieren der Extension ausschließlich Dateien aus dem **Stammordner**. Die Unterordner `Dokumentation/` und `Hilfsskripte/` werden zur Laufzeit nicht benötigt und können ohne Auswirkung auf den Betrieb verschoben oder entfernt werden.

3.2 Extension-Manifest (`extension.toml`)

Die Datei `extension.toml` ist die **Visitenkarte** der Extension: Sie teilt dem Isaac-Sim-Extension-Manager mit, wie die Extension heißt, welche Version sie hat und welches Python-Modul als Einstiegspunkt dient.

Häufigste Fehlerquelle beim Laden der Extension: Wird eine Extension im Extension-Manager nicht gefunden oder lässt sich nicht korrekt laden, liegt das Problem fast immer in `extension.toml` — entweder stimmt der Dateinhalt (Modulname, Pfad) nicht, oder der Ordner, der `extension.toml` enthält, ist im Extension-Manager nicht als Suchpfad eingetragen.

```

1 [package]
2 title      = "Dobot Digital Twin"
3 description = "ROS 2 Treiber und UI fuer den Dobot"
```

```

4 version      = "1.0.0"
5 authors      = ["Tobi"]
6
7 [dependencies]
8 "omni.kit.uiapp" = {}
9 "omni.ui"       = {}
10
11 [[python.module]]
12 name = "dobot_ros"
13 path = "."

```

Listing 1: extension.toml

Der Eintrag `[[python.module]]` teilt Isaac Sim mit, welches Python-Modul geladen werden soll und wo es liegt. Beim Aktivieren der Extension importiert Isaac Sim das Modul `dobot_ros` aus dem Verzeichnis `.` (dem Extension-Stammordner selbst). Python lädt dabei zuerst `__init__.py`, die nur eine einzige Zeile enthält:

```

1 from .extension import DobotBridgeExtension

```

Damit steht `DobotBridgeExtension` im Namespace des Moduls zur Verfügung. Isaac Sim durchsucht den Namespace automatisch nach einer Klasse, die `omni.ext.IExt` implementiert, und findet `DobotBridgeExtension`. Anschließend ruft das Kit-Framework **zwei Lifecycle-Methoden** auf:

<code>on_startup(ext_id)</code>	Wird aufgerufen, sobald die Extension aktiviert wird (Häkchen im Extension-Manager oder Autostart). Hier werden Zustandsvariablen initialisiert, das UI-Fenster gebaut und der Frame-Update-Callback registriert.
<code>on_shutdown()</code>	Wird aufgerufen, wenn die Extension deaktiviert oder Isaac Sim beendet wird. Trennt die Dobot-Verbindung, stoppt alle Hintergrundthreads und gibt UI-Ressourcen frei.

Solange die Extension nicht korrekt als `omni.ext.IExt` erkannt wird — etwa weil `extension.toml` den falschen Modulnamen oder Pfad enthält — werden weder `on_startup` noch `on_shutdown` jemals aufgerufen, und die Extension erscheint im Manager als nicht ladbar.

3.3 Installations-Hilfsskript (install_extension.py)

Wozu dient das Skript? Isaac Sim kennt zwei Wege, eine Extension zu finden:

1. **Suchpfad im Extension-Manager** (Entwicklung): Im Extension-Manager unter *Fenster* → *Extensions* → *Zahnrad* → *Paths* kann der Quellordner direkt eingetragen werden.
2. **Roaming-Extensions-Verzeichnis** (Installation): Isaac Sim durchsucht beim Start automatisch `%APPDATA%\NVIDIA Corporation\Omniverse\Extensions`. Wer die Extension dauerhaft ohne manuellen Pfadeintrag betreiben oder auf einem Produktionsrechner einsetzen möchte, kopiert den Ordner dorthin. Genau das erledigt `install_extension.py`.

Wann wird das Skript benötigt?

- Deployment auf einem Rechner ohne Entwicklungsumgebung
- Saubere Installation ohne Git-Artefakte und Cache-Dateien
- Schnelle Ersteinrichtung auf einem neuen System

Verwendung

```

1 # Standard-Installation in das Omniverse-Roaming-Verzeichnis:
2 python install_extension.py
3
4 # Eigenes Zielverzeichnis und Ordnername:
5 python install_extension.py --dest "C:\MeinPfad" --name dobot_ros
6
7 # Vorschau: zeigt Quelle und Ziel, kopiert aber nichts:
8 python install_extension.py --dry-run

```

Listing 2: install_extension.py – Verwendung

3.4 Erstinstallation

1. **Extension-Pfad eintragen:** Im Extension-Manager unter *Fenster* → *Extensions* → *Zahnrad* → *Paths* den Ordner `dobot_ros` als Suchpfad hinzufügen. Alternativ `install_extension.py` ausführen, um die Extension in das Roaming-Verzeichnis zu kopieren.
2. **Extension aktivieren:** Im Extension-Manager nach `dobot_ros` suchen und aktivieren.
3. **Pakete installieren:** Im Extension-Panel *Install packages* klicken. Dies installiert `pydobot` und `pyserial` in der Isaac-Sim-Python-Umgebung via `omni.kit.pipapi`. Danach Extension deaktivieren und neu aktivieren.

4 Robotermodell

Bevor die Architektur und Implementierung der Extension beschrieben werden, ist ein Verständnis des Robotermodells notwendig: Ohne ein korrekt importiertes Modell kann die Extension nicht mit der Simulationsgelenkstruktur kommunizieren.

4.1 URDF-Import

URDF (*Unified Robot Description Format*) ist ein XML-basiertes Format zur vollständigen Beschreibung eines Roboters: Körper (Links), Gelenke (Joints), 3D-Meshes, Trägheitsmomente und Kollisionsgeometrien. Isaac Sim kann URDF-Dateien direkt importieren und daraus ein simulierbares USD-Modell erzeugen.

Ohne ein korrekt importiertes Modell gibt es in Isaac Sim keine Gelenkstruktur, auf die die Dynamic-Control-API zugreifen kann. Die Extension setzt voraus, dass das Modell als **Artikulation** geladen ist – d. h. `base_link` trägt die `ArticulationRootAPI` und alle Gelenke sind als DOFs (Degrees of Freedom) registriert. Nur dann liefert `get_articulation()` einen gültigen Handle.

Artikulation

In Isaac Sim bezeichnet *Artikulation* eine zusammenhängende Kette von starren Körpern, die über Gelenke verbunden sind und gemeinsam von der Physik-Engine simuliert werden. Der Prim, der `ArticulationRootAPI` trägt, ist der Wurzelknoten dieser Kette – alle Kindknoten mit Gelenken gehören automatisch dazu. Ohne dieses Schema behandelt Isaac Sim die einzelnen Meshes als unabhängige Starrkörper ohne kinematische Verbindung.

DOF (Degree of Freedom)

Jedes bewegliche Gelenk in der Artikulation bekommt beim Start der Physik-Simulation eine interne Indexnummer – seinen *DOF-Index*. Über diesen Index spricht die Dynamic-Control-API das Gelenk an: Winkel

lesen, Sollwert setzen. „Als DOF registriert“ bedeutet, dass das Gelenk von der Physik-Engine erkannt wurde und einen gültigen Index hat.

Handle

Ein *Handle* ist ein numerischer Bezeichner, den `get_articulation()` zurückgibt, sobald die Physik läuft und die Artikulation gefunden wurde. Er ist vergleichbar mit einem Datei-Handle beim Öffnen einer Datei: Erst wenn er gültig ist ($\neq$ `INVALID_HANDLE`), können alle weiteren DC-API-Aufrufe auf den Roboter zugreifen. Deshalb initialisiert die Extension die DC-Verbindung lazy – erst nach dem Drücken von *Play*.

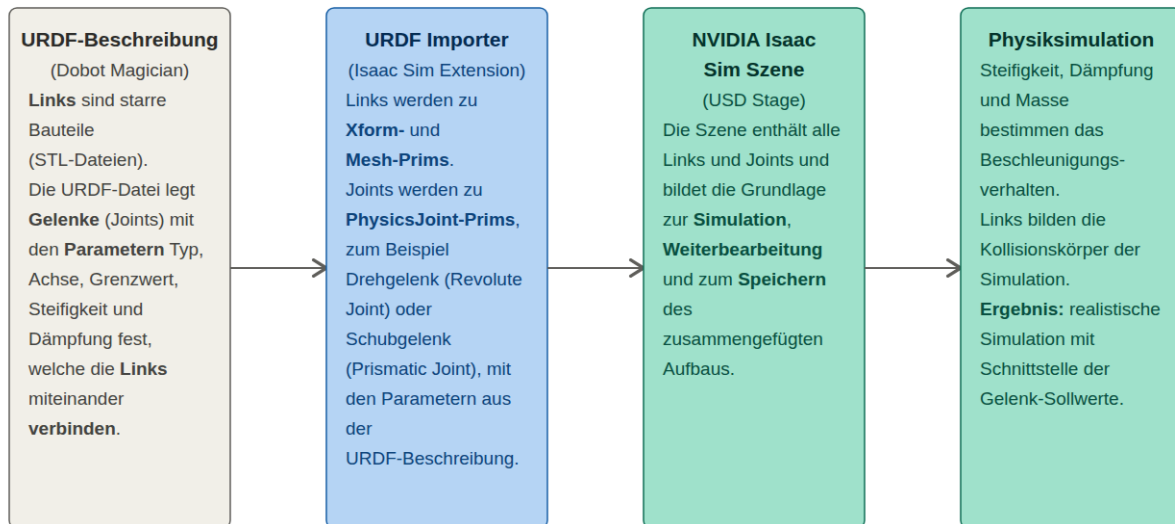


Abbildung 2: Importpipeline vom URDF zum physikfähigen Modell: Die URDF-Beschreibung enthält Links (STL-Meshes) und Joints mit Achsen, Grenzen und Antriebsparametern. Der Isaac-Sim-URDF-Importer erzeugt daraus Xform-/Mesh-Prims und PhysicsJoint-Prims in der USD-Stage. Erst dann liefert `get_articulation()` einen gültigen Handle für die DC-API.

4.2 Gelenkstruktur des Dobot Magician

Der Dobot Magician hat ein Parallelogrammgestänge im Inneren, das kinematisch dafür sorgt, dass `link_6` (Endeffektor-Flansch) stets horizontal bleibt. Im URDF wird dieses Gestänge *nicht* als eigene Links modelliert – stattdessen bildet die Kette direkt:

$$\begin{array}{ccccccc}
 \text{base_link} & \xrightarrow{\text{joint_1 (Z, } \pm 180^\circ)} & \text{link_1} & \xrightarrow{\text{joint_2 (Z, } 0-90^\circ)} & \text{link_2} & \xrightarrow{\text{joint_5 (-Z, } 0-120^\circ)} & \text{link_5} \\
 & & \text{link_5} & \xrightarrow{\text{joint_6 (-Z, } \pm 180^\circ)} & \text{link_6} & \xrightarrow{\text{joint_suction (fixed, } +4 \text{ cm)}} & \text{link_suction}
 \end{array}$$

`joint_3` und `joint_4` existieren im URDF nicht – das Parallelogramm ist kinematisch in die Winkelformeln von `joint_5` und `joint_6` eingerechnet (siehe Abschnitt 6).

5 Systemarchitektur

Mit dem Robotermodell als Grundlage lässt sich nun die Gesamtarchitektur der Extension beschreiben. Diese gliedert sich in vier klar voneinander getrennte Schichten, die von oben (UI) bis unten (Hardware) aufeinander aufbauen.

5.1 Schichtenmodell

Schicht	Klassen / Komponenten	Technologie
UI-Schicht	<code>DobotBridgeExtension</code> : Fenster, Tabs, Callbacks	<code>omni.ui</code>
Simulations-Schicht	Dynamic-Control-API, USD-Stage, <code>omni.timeline</code>	<code>omni.isaac.*</code>
Middleware-Schicht	ROS2-Node, Publisher, <code>/joint_states</code>	<code>rclpy</code> (Humble)
Hardware-Schicht	<code>DobotROSNode</code> : Pose-Abfrage, Bewegungsbefehle	<code>pydobot</code> , <code>pyserial</code>

5.2 Hauptklassen

Die gesamte Logik verteilt sich auf zwei Klassen, die klar voneinander getrennte Verantwortlichkeiten haben:

<code>DobotROSNode</code>	Kapselt die serielle USB-Verbindung zum echten Dobot-Roboter über <code>pydobot</code> sowie den ROS2-Node und den JointState-Publisher. Enthält alle Hardware-nahen Methoden: Pose-Abfrage, Kinematikberechnung, Bewegungsbefehle, Vakuumgreifer und RC-Servo-Ansteuerung.
<code>DobotBridgeExtension</code>	Subklasse von <code>omni.ext.IExt</code> . Verwaltet das Omniverse-UI-Fenster mit zwei Tabs, die Isaac-Sim-Dynamic-Control-Kopplung, den Frame-Update-Loop sowie die Hintergrundthreads für Pick-&-Place-Sequenzwiedergabe und Würfel-Tracking.

5.3 Datenfluss

5.3.1 Richtung 1: Echter Roboter → Simulation (Digital Twin)

1. Pro Frame ruft `DobotBridgeExtension._on_update()` die Methode `DobotROSNode.update_step()` auf.
2. `update_step()` liest via `device.pose()` die aktuelle Pose: $(x, y, z, r, j_1, j_2, j_3, j_4)$ in mm und Grad.
3. Kinematische Transformation liefert Isaac-Sim-kompatible Gelenkwinkel.
4. Konvertierung nach Radiant; Publikation als `JointState` auf `/joint_states`.
5. `_drive_joints()` setzt Winkel über DC-API auf die Isaac-Sim-Artikulation.
6. Status-Dashboard im UI aktualisieren.

5.3.2 Richtung 2: Simulation / UI → Echter Roboter

1. Nutzer betätigt eine Steuertaste (z. B. $X + 5\text{ mm}$).
2. UI-Callback wird ausgelöst (z. B. `_on_jog_clicked(5.0, 0.0)`).
3. `DobotROSNode.jog(dx, dy)` wird aufgerufen.
4. Aktuelle Pose einlesen, Delta addieren, `move_to()` an Dobot senden.
5. Nächster Frame liest neue Position – Regelkreis geschlossen.

6 Kinematisches Modell

Bevor die einzelnen Implementierungskomponenten betrachtet werden, ist das kinematische Modell zu verstehen, da es den Kern der Datenverarbeitung im Digital-Twin-Betrieb bildet.

6.1 Koordinatenkonventionen (pydobot)

Die Bibliothek `pydobot` liest vom Dobot Magician über das serielle Protokoll (Kommando-ID 10) die aktuelle Pose: $(x, y, z, r, j_1, j_2, j_3, j_4)$. Dabei gilt folgende Konvention:

j_1 : Basis-Rotation (Yaw, Z-Achse) [Grad] (1)

j_2 : Oberarm-Pitch, absoluter Weltwinkel von Horizontal [Grad] (2)

j_3 : Unterarm-Pitch, absoluter Weltwinkel von Horizontal [Grad] (3)

j_4 : Servo-gesteuerte Endeffektordrehung (Wristgelenk) [Grad] (4)

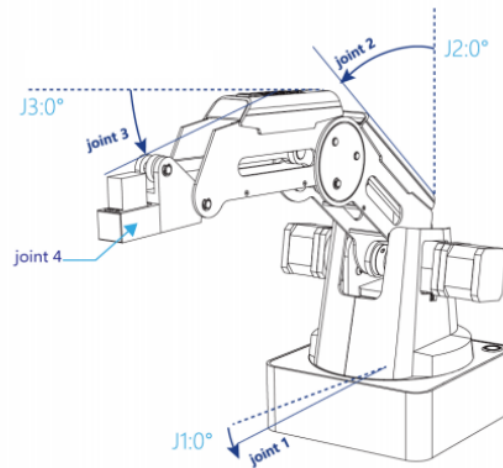


Abbildung 3: Gelenkwinkelkonvention des Dobot Magician. J1 rotiert um die Z-Achse der Basis (Yaw), J2 und J3 sind Pitchgelenke mit Nulllage in der Horizontalen, J4 dreht den Endeffektor. Im URDF dieses Projekts fehlen J3 und J4 als eigenständige Links – ihre kinematische Wirkung ist in die Formeln für `joint_5` und `joint_6` eingerechnet.

Durch das Parallelogrammgestänge des Dobot Magician gilt näherungsweise $j_3 \approx 0^\circ$ (Unterarm bleibt in Weltkoordinaten horizontal), sofern das Gestänge korrekt eingestellt ist.

6.2 Isaac-Sim-Gelenkstruktur

Das USD-Modell unter `/World/magician/base_link` definiert vier aktive Revolute-Gelenke:

DOF-Index	USD-Name	Kinematische Bedeutung	Physischer Bereich
0	<code>joint_1</code>	Basis-Yaw	$\pm 180^\circ$
1	<code>joint_2</code>	Schulter-Pitch	$0^\circ - 90^\circ$
2	<code>joint_5</code>	Parallelogramm-Kompensation	$0^\circ - 120^\circ$
3	<code>joint_6</code>	Endeffektor-Vertikalisierung	$\pm 180^\circ$

6.3 Einheitenstrategie

Alle internen Berechnungen verwenden **Grad**. Radiant-Konvertierung findet ausschließlich an den API-Grenzen statt:

Kontext	Einheit	Konvertierung
Interne Kinematik	Grad	–
DC-API <code>set_dof_state.pos</code>	Radiant	<code>math.radians()</code>
DC-API <code>set_dof_position_target</code>	Radiant	<code>math.radians()</code>
ROS2 <code>JointState.position</code>	Radiant	<code>math.radians()</code>
UI-Dashboard-Anzeige	Grad	–

6.4 Berechnungsformeln

6.4.1 joint_1 – Gierwinkel mit festem Offset

Der Gierwinkel j_1 des Dobot wird mit einem hardcodierten Offset von $\Delta_{\text{Yaw}} = -45^\circ$ korrigiert, um die Ausrichtung der Simulationsachse gegenüber dem realen Roboter auszugleichen:

$$\theta_1 = j_1 + \Delta_{\text{Yaw}} = j_1 - 12^\circ \quad (5)$$

Der Wert ist als Modul-Konstante `J1_YAW_OFFSET_DEG = -12.0` in `extension.py` festgelegt.

6.4.2 joint_2 – Direkte Übernahme

$$\theta_2 = j_2 \quad [^\circ] \quad (6)$$

6.4.3 joint_5 – Parallelogramm-Kompensation

Der Parallelogramm-Mechanismus hält `link_5` in Weltkoordinaten horizontal. j_3 und j_2 sind beide absolute Weltwinkel; `joint_5` muss ihre Differenz kodieren:

$$\theta_5 = j_3 - j_2 \quad [^\circ] \quad (7)$$

```

1 # Parallelogramm-Kompensation: Unterarm in Weltlage halten
2 pos_j5 = j3 - j2 # Grad

```

Listing 3: Berechnung `joint_5` in `update_step()`

6.4.4 joint_6 – Endeffektor-Vertikalisierung

Anforderung: `link_6` (Saugnapf-Träger) soll stets senkrecht nach unten zeigen, unabhängig von der Armkonfiguration.

Herleitung: Die Summe aller Pitch-Rotationen entlang der kinematischen Kette muss in Weltkoordinaten 90° ergeben:

$$\theta_2 + \theta_5 + \theta_6 = 90^\circ \quad (8)$$

Einsetzen von Gleichung (7):

$$\begin{aligned}
 j_2 + (j_3 - j_2) + \theta_6 &= 90^\circ \\
 j_3 + \theta_6 &= 90^\circ
 \end{aligned}$$

Berücksichtigung der Servo-Verdrehung j_4 :

$$\theta_6 = 90^\circ - j_3 - j_4 \quad (9)$$

Wichtiger Hinweis: Der Term j_2 tritt in Gleichung (9) *nicht* auf – er kürzt sich heraus. Zwei Entwicklungsfehler wurden behoben:

1. `pos_j6 = j2 + j3 - j4` ließ θ_6 mit der Schulterposition mitlaufen → Endeffektor kippte bei angehobenem Arm nach oben.
2. Eine Mischung von $\frac{\pi}{2}$ (Radiant) und Grad-Werten in einer einzigen Formel führte zu Einheitenfehlern.

```

1 # Kette: j2 + (j3-j2) + j6 = 90  =>  j6 = 90 - j3 - j4
2 # j2 darf hier NICHT auftreten
3 pos_j6 = 90.0 - j3 - j4      # Grad

```

Listing 4: Korrekte Berechnung `joint_6` in `update_step()`

6.5 Vollständige Kinematik in `update_step()`

```

1 pose = self.device.pose() # (x, y, z, r, j1, j2, j3, j4)
2 if pose is not None and len(pose) >= 8:
3     x, y, z, r = pose[0], pose[1], pose[2], pose[3]
4     j1, j2, j3, j4 = pose[4], pose[5], pose[6], pose[7]
5
6     # Gierwinkel-Offset: Simulationsachse gegenueber Realroboter
7     j1 = j1 + J1_YAW_OFFSET_DEG # -12 Grad (kalibriert)
8
9     # Parallelogramm: Unterarm in Weltlage halten
10    pos_j5 = j3 - j2
11    # Endeffector vertikal: j2+(j3-j2)+j6=90 => j6=90-j3-j4
12    pos_j6 = 90.0 - j3 - j4
13
14    joints_deg = [j1, j2, pos_j5, pos_j6]
15    self.last_valid_positions = joints_deg
16    self.last_pose = {'x': x, 'y': y, 'z': z, 'r': r, 'j1': j1}
17
18    # ROS2-Konvention: Radiant
19    msg.position = [math.radians(v) for v in joints_deg]
20    self.publisher_.publish(msg)

```

Listing 5: Kinematik-Kern in `update_step()` (Ausschnitt aus `extension.py`)

7 Implementierung

Dieser Abschnitt beschreibt die technischen Details der Implementierung: wie Module geladen werden, wie die beiden Hauptklassen intern aufgebaut sind, wie die Dynamic-Control-Kopplung funktioniert und wie das Threading-Modell aufgebaut ist.

7.1 Lazy-Import-System

ROS2 (`rclpy`) und `pydobot` werden nicht beim Extension-Start importiert, sondern erst beim ersten Klick auf *Connect*. Dieser Mechanismus verfolgt zwei Ziele:

1. Die Extension lädt fehlerfrei, auch wenn ROS2 oder `pydobot` noch nicht installiert sind.
2. Teure Import-Operationen verzögern den Isaac-Sim-Start nicht.

Einmal erfolgreich geladene Module werden in Modul-globalen Variablen gecacht:

```

1 _rclpy      = None      # rclpy-Modul
2 _Node       = None      # rclpy.node.Node
3 _JointState = None      # sensor_msgs.msg.JointState
4 _Dobot      = None      # pydobot.Dobot
5
6 def _try_import_ros():
7     global _rclpy, _Node, _JointState
8     if _rclpy is not None:
9         return True          # Bereits gecacht
10    try:
11        import rclpy
12        from rclpy.node import Node
13        from sensor_msgs.msg import JointState
14    except Exception:
15        return False          # ROS2-Bridge nicht verfuegbar
16    _rclpy, _Node, _JointState = rclpy, Node, JointState
17    return True
18
19 def _try_import_dobot():
20     global _Dobot
21     if _Dobot is not None:
22         return True
23     try:
24         from pydobot import Dobot
25     except Exception:
26         return False          # pydobot nicht installiert
27     _Dobot = Dobot
28     return True

```

Listing 6: Lazy-Import-Caches und `_try_import_ros()`

7.2 Klasse DobotROSNode

DobotROSNode ist die Hardware-Schicht der Extension. Sie kapselt die serielle Verbindung zum Dobot und den ROS2-Node.

7.2.1 Initialisierung (`__init__`)

Der Konstruktor führt in dieser Reihenfolge aus:

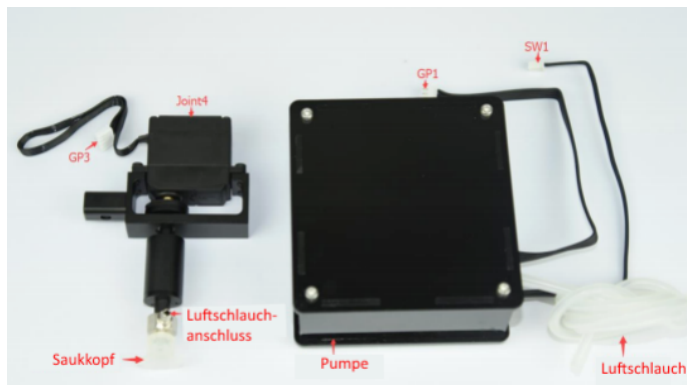
1. Lazy-Imports prüfen; bei Fehler `RuntimeError` mit beschreibender Meldung.
2. ROS2-Kontext initialisieren (`rclpy.init()`), idempotent durch `rclpy.ok()`-Guard).
3. ROS2-Node `dobot_extension_bridge` erstellen.
4. JointState-Publisher auf `/joint_states` anlegen (Queue-Tiefe 10).
5. Serielle Verbindung via `pydobot.Dobot(port=com_port)`.
6. Fallback-Puffer `last_valid_positions = [0.0, 0.0, 0.0, 0.0]`.

7.2.2 Methoden-Referenz

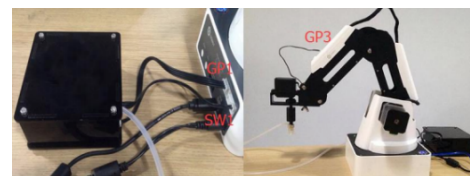
Methode	Beschreibung
<code>update_step()</code>	Liest Pose vom Dobot, berechnet Gelenkwinkel, publiziert <code>JointState</code> , ruft <code>rclpy.spin_once(timeout=0)</code> auf (nicht-blockierend). Gibt (<code>pose_dict</code> , <code>joints_deg_list</code>) zurück. Bei Fehler: Fallback auf <code>last_valid_positions</code> .
<code>set_suction(active)</code>	Schaltet Vakuumgreifer SW1 (Kommando ID 62). Fallback-Liste über mehrere pydobot-Methodennamen (<code>suck</code> , <code>set_end_effector_suction</code> , <code>set_vacuum_gripper</code>).
<code>move_vertical(delta_mm)</code>	Liest aktuelle Pose, addiert <code>delta_mm</code> auf Z, sendet <code>move_to</code> . Positiv = aufwärts.
<code>jog(dx, dy)</code>	Horizontaler Versatz in der X-Y-Ebene. Liest aktuelle Pose, addiert (<code>dx</code> , <code>dy</code>) in mm.
<code>go_home()</code>	Fährt zu HOME_X/Y/Z/R aus <code>constants.py</code> .
<code>set_servo_gp3(angle_deg)</code>	RC-Servo an GP3-PWM1 via Dobot-Rohprotokoll (Kommandos ID 130 + ID 132). Winkel $[0^\circ, 180^\circ]$, linearer Duty-Cycle.
<code>play_sequence(poses)</code>	Einfache Sequenz-Wiedergabe ohne Sicherheits-Anfahrt (interne Fallback-Methode).
<code>stop_sequence()</code>	Leert Dobot-interne Queue via <code>_set_queued_cmd_stop_exec</code> , <code>_set_queued_cmd_clear</code> , <code>_set_queued_cmd_start_exec</code> .
<code>shutdown()</code>	Schließt COM-Port via <code>device.close()</code> , zerstört ROS2-Node via <code>destroy_node()</code> .

7.2.3 Hardware: Pneumatischer Sauggreifer

Der Sauggreifer besteht aus zwei getrennten Einheiten, die gemeinsam von `set_suction()` gesteuert werden.



(a) Saugkopf mit Luftschlauch an GP3/Joint 4 (links) und Pumpeneinheit mit den Anschlüssen GP1 und SW1 (rechts). `set_suction(True)` schaltet SW1, was die Pumpe aktiviert.



(b) Verkabelung: Pumpe über GP1 und SW1 (links), Saugkopf über GP3 am vierten Gelenk (rechts).

Abbildung 4: Pneumatischer Sauggreifer des Dobot Magician – Komponenten und Anschlusschema. Der Fallback-Mechanismus in `set_suction()` deckt verschiedene pydobot-Versionsnamen für den Saugbefehl ab.

7.2.4 Fehlerbehandlung und Robustheit

Kommunikationsunterbrechung: `update_step()` fängt alle Ausnahmen auf. Bei Fehler werden die Werte aus `last_valid_positions` verwendet – das Simulationsmodell *bleibt in der letzten bekannten Stellung*.

pydobot-Versionsinkompatibilität:

```

1 for name, args in [
2     ('suck', (active,)),
3     ('set_end_effector_suction', (active,)),
4     ('set_vacuum_gripper', (1 if active else 0,)),
5 ]:
6     fn = getattr(self.device, name, None)
7     if callable(fn):
8         return fn(*args)
9 raise RuntimeError('Keine Saugbefehl-Methode in pydobot gefunden.')
```

Listing 7: Suction-Fallback-Mechanismus in `set_suction()`

7.3 Dynamic-Control-Kopplung

Die Dynamic-Control-API verbindet die kinematischen Werte aus `DobotROSNode` mit der Gelenkstruktur in der Simulation.

Initialisierungsproblem: Die Isaac-Sim-Dynamic-Control-API benötigt eine *laufende* Physik-Simulation. Vor dem Drücken von *Play* liefert `get_articulation()` `INVALID_HANDLE`.

Lazy-Initialisierung:

```

1 if self._art_init_pending and self._dc is None:
2     import omni.timeline
3     if omni.timeline.get_timeline_interface().is_playing():
4         robot_path = self._robot_path_input.model \
5             .get_value_as_string().strip()
6         self._init_articulation(robot_path, silent=True)
7         if self._dc is not None:
8             self._art_init_pending = False
```

Listing 8: Lazy DC-Init in `_on_update()`

DOF-Handles abrufen:

```

1 joint_names = ['joint_1', 'joint_2', 'joint_5', 'joint_6']
2 for jname in joint_names:
3     dof = dc.find_articulation_dof(art, jname)
4     self._dof_handles.append(
5         dof if dof != dc_mod.INVALID_HANDLE else None
6     )
7 found = sum(1 for d in self._dof_handles if d is not None)
8 # Status: "Modell verbunden: 4/4 DOFs gefunden."
```

Listing 9: DOF-Handle-Abruf in `_init_articulation()`

Gelenkwinkel setzen – `_drive_joints()`: Pro Gelenk werden zwei komplementäre DC-Aufrufe verwendet:

```

1 for dof_handle, angle_deg in zip(self._dof_handles, joints_deg):
2     if dof_handle is None:
3         continue
4     rad = math.radians(float(angle_deg)) # Grad -> Radian
5
6     # 1) Kinematischen Zustand direkt setzen
7     state = self._dc_mod.DofState()
8     state.pos = rad
9     state.vel = 0.0
10    self._dc.set_dof_state(dof_handle, state, self._dc_mod.STATE_POS)
11
12    # 2) PhysX-Drive-Ziel setzen (Pflicht: [-2pi, 2pi])
13    self._dc.set_dof_position_target(dof_handle, rad)

```

Listing 10: `_drive_joints()` – doppelter DC-Aufruf

`set_dof_position_target()` erzwingt $[-2\pi, 2\pi]$. Die Kinematik-Formeln halten alle Ausgaben innerhalb dieses Bereichs.

Die `ArticulationRootAPI` liegt beim importierten Dobot-Modell auf `/World/magician/base_link` – *nicht* auf dem Elternprim `/World/magician`. Der vorausgefüllte Standardwert im UI ist entsprechend gesetzt.

7.4 Klasse DobotBridgeExtension

`DobotBridgeExtension` ist die oberste Schicht der Extension. Sie verbindet UI, Simulation und die Hardware-Klasse `DobotROSNode` und verwaltet alle Hintergrundthreads.

7.4.1 Lebenszyklusmethoden

`on_startup(ext_id)` Alle Zustandsvariablen initialisieren; UI-Fenster (490×590 px) aufbauen; Docking-Task starten.

`_dock_window_deferred()` Async-Coroutine: 2 Frames warten, Fenster neben Property-Panel docken via `ui.DockPosition.SAME`. Zwei Frames notwendig, da Fenster erst nach erstem Render im Workspace-Index erscheint.

`on_shutdown()` Dock-Task canceln (verhindert „extension object is still alive“-Warnung), `_cleanup()` aufrufen, Fenster zerstören.

7.4.2 Frame-Update-Ablauf (`_on_update()`)

Dieser Callback wird pro Frame vom Isaac-Sim-Event-System aufgerufen und ist der zentrale Takt der Digital-Twin-Kopplung:

```

1 def _on_update(self, event):
2     if not self._ros_node:
3         return
4
5     # 1. Lazy DC-Initialisierung
6     if self._art_init_pending and self._dc is None:
7         import omni.timeline
8         if omni.timeline.get_timeline_interface().is_playing():
9             robot_path = self._robot_path_input.model \

```

```

10         .get_value_as_string().strip()
11     self._init_articulation(robot_path, silent=True)
12     if self._dc is not None:
13         self._art_init_pending = False
14
15     # 2. Dobot-Daten lesen und ROS2 publizieren
16     pose, joints = self._ros_node.update_step()
17
18     # 3. DC-Kopplung (nur wenn Simulation laeuft)
19     import omni.timeline
20     if omni.timeline.get_timeline_interface().is_playing():
21         self._drive_joints(joints)
22
23     # 4. Dashboard aktualisieren
24     self._update_display(pose, joints)

```

Listing 11: `_on_update()` – vollständiger Ablauf

7.4.3 Ressourcenfreigabe (`_cleanup()`)

1. Stop-Flag Wiedergabe-Thread setzen; `join(timeout=1 s)`.
2. Stop-Flag Tracking-Thread setzen; `join(timeout=1 s)`.
3. Würfel-Prims aus USD-Stage entfernen.
4. Update-Subscription freigeben.
5. DC-Interface und Handles loslassen.
6. `DobotROSNode.shutdown()` aufrufen (COM-Port schließen, ROS2-Node zerstören).

7.5 Threading-Modell

Die Extension verwendet drei Threads, um blockierende Operationen vom Isaac-Sim-Hauptthread zu entkoppeln:

Thread	Name	Aufgabe
Hauptthread	(Isaac Sim)	UI-Callbacks, Frame-Updates (<code>_on_update</code>), DC-API-Aufrufe
Wiedergabe	<code>dobot_play</code>	Blockierende <code>wait_for_cmd()</code> -Schleife
Tracking	<code>dobot_cube_track</code>	Periodische <code>move_to</code> -Befehle für Würfel

Begründung für Threading statt `asyncio`: `pydobot.wait_for_cmd()` ist eine blockierende Polling-Schleife. Im `asyncio`-Event-Loop würde sie den gesamten Isaac-Sim-Hauptthread einfrieren. Im Hintergrundthread blockiert sie nur diesen Thread; Isaac Sim läuft ungehindert weiter.

Thread-Sicherheit: `pydobot`s internes `RLock` serialisiert alle seriellen Zugriffe. Statusschreibzugriffe aus Hintergrundthreads (`_set_status()`) sind durch den GIL von CPython atomisch.

Graceful Shutdown: Beide Stop-Flags werden in `_cleanup()` gesetzt; danach `join(timeout=1 s)` pro Thread.

8 Benutzeroberfläche

Das UI-Fenster wird beim Start der Extension aufgebaut und automatisch neben dem Property-Panel gedockt. Es besteht aus zwei Tabs und einem dauerhaft sichtbaren Status-Dashboard.

8.1 Tab 0: Steuerung

UI-Element	Funktion
COM-Port-Feld	Serieller Port (Standard: COM3)
Connect-Button	Verbindungsaufbau; zeigt nach Erfolg grün ●
Disconnect-Button	Verbindungstrennung und UI-Reset
Verbindungsstatus (●)	Rot = getrennt, grün = verbunden
Robot-USD-Feld	USD-Pfad der Artikulations-Root in der Szene; Standard <code>/World/magician/base_link</code>
Install packages	Installiert <code>pydobot</code> und <code>pyserial</code> via <code>omni.kit.pipapi</code>
Suction ON/OFF	Vakuumgreifer umschalten
Up / Down 10 mm	Vertikalbewegung ± 10 mm
X $\pm$ / Y $\pm$ 5 mm	Horizontaler JOG ± 5 mm
Servo GP3 $\pm 10^\circ$	Servo-Winkel erhöhen / verringern
Würfel spawnen / entfernen	Ziel-Würfel ein-/ausblenden; funktioniert auch ohne COM-Port
Rate-Slider (0,5–10 Hz)	Tracking-Frequenz konfigurieren

8.2 Tab 1: Pick & Place

UI-Element	Funktion
Go Home	Roboter zur Nullposition fahren
Servo GP3 $\pm 10^\circ$	Identisch zu Tab 0
Saugen (Aufn.) EIN/AUS	Virtueller Sauger-Toggle für Aufzeichnung; schaltet Zustand um <i>und</i> speichert automatisch Wegpunkt
Rec Punkt	Aktuellen Wegpunkt manuell aufzeichnen
<i>n</i> Pkt.	Zähler der gespeicherten Wegpunkte
Löschen	Alle Wegpunkte und Saugzustand zurücksetzen
Play	Pick-&-Place-Sequenz im Hintergrundthread starten
Stop	Laufende Sequenz abbrechen und Dobot-Queue leeren
Scrollbare Waypoint-Liste	Zeigt alle Punkte mit Koordinaten und Saugzustand; Saugpunkte in Grün

8.3 Status-Dashboard (immer sichtbar)

Element	Beschreibung
Status-Label	Farbkodierter Betriebsstatus: grün = OK / Verbunden, gelb = Pending / Warnung, rot = Fehler
ROS-Topic-Label	ROS topic: /joint_states
Pose-Label	Kartesische Position: Pose: x=... y=... z=... r=...
Joints-Label	Gelenkwinkel in Grad: Joints [°]: j1=... j2=... j5=... j6=...

9 Funktionen

Dieser Abschnitt beschreibt die beiden Hauptfunktionen der Extension, die über die einfache Fernsteuerung hinausgehen: das Pick-&-Place-Szenario und das Würfel-Tracking. Abschließend folgt eine Schritt-für-Schritt-Anleitung für den täglichen Betrieb.

9.1 Pick-&-Place-Szenario (Teaching)

9.1.1 Datenstruktur der Wegpunkte

Jeder Wegpunkt ist ein 5-Tupel: $(x_i, y_i, z_i, r_i, s_i) \in \mathbb{R}^4 \times \{0, 1\}$ mit (x, y, z) in mm, r in Grad, s = Saugzustand (0 = aus, 1 = ein).

9.1.2 Aufzeichnung

Rec Punkt Speichert `DobotROSNode.last_pose` + aktuellen virtuellen Saugzustand als neuen Wegpunkt. *Kein Hardware-Befehl* – der Saugzustand wird erst bei der Wiedergabe ausgeführt.

Saugen EIN/AUS Wechselt den virtuellen Saugzustand *und* speichert automatisch einen Wegpunkt mit dem neuen Zustand. Ermöglicht präzises Einteachen von Greif- und Ablegepositionen.

Löschen Setzt Wegpunktliste, Saugzustand und Zähler zurück.

9.1.3 Wiedergabe

Die Wiedergabe läuft in einem dedizierten Hintergrundthread (`dobot_play`), damit die blockierende `wait_for_cmd()`-Schleife den Isaac-Sim-Hauptthread nicht einfriert.

```

1 APPROACH_MM = 10.0    # 1 cm Sicherheitsabstand von oben
2
3 for i, (x, y, z, r, suction_active) in enumerate(poses):
4     if self._stop_play_flag: break
5     suction_changes = (suction_active != prev_suction)
6
7     if suction_changes:
8         # 1 cm ueber Zielpunkt anfahren (kein seitliches Streifen)
9         _move_and_wait(x, y, z + APPROACH_MM, r)
10
11     # Eigentliche Zielposition anfahren
12     _move_and_wait(x, y, z, r)
13
14     # Saugzustand schalten (nach Positionserreichen)

```

```

15     self._ros_node.set_suction(suction_active)
16     _time.sleep(0.5)      # Druckaufbau/-abbau abwarten
17
18     if suction_changes:
19         _move_and_wait(x, y, z + APPROACH_MM, r)
20
21     prev_suction = suction_active

```

Listing 12: Kernlogik `_play_sequence_thread()` (vereinfacht)

Nach dem letzten Wegpunkt oder nach Abbruch fährt der Roboter automatisch zur Nullposition zurück und der Sauger wird deaktiviert. `_on_stop_sequence_clicked()` setzt das Stop-Flag und leert die interne Dobot-Bewegungs-Queue sofort.

9.2 Ziel-Würfel-Tracking

Der Ziel-Würfel ist ein orangenes 2,5 cm-Objekt in der Isaac-Sim-Szene, das frei verschoben werden kann. Die Tracking-Funktion liest seine Position aus der USD-Stage und sendet entsprechende Fahrbefehle an den Dobot. Das Tracking funktioniert auch ohne verbundenen COM-Port – in diesem Fall wird die Zielposition nur angezeigt.

9.2.1 Würfel erstellen (`_spawn_target_cube()`)

1. Alten Würfel entfernen (Idempotenz).
2. Startposition aus `DobotROSNode.last_pose` (mm → m: Division durch 1000); Fallback auf Home-Position wenn nicht verbunden.
3. `UsdGeom.Cube`, Kantenlänge 2,5 cm, unter `/World/DobotTargetCube`.
4. Oranges `UsdPreviewSurface`-Material (diffuse RGB (1.0,0.45,0.0), Opacity 0,9).
5. Tracking-Thread starten.

9.2.2 Tracking-Schleife

Statt des Würfelzentrums wird der Normalenvektor von Würfelzentrum zu aktueller TCP-Position berechnet und der TCP auf die dem Roboter zugewandte Würfelfläche ausgerichtet:

```

1 CUBE_HALF_MM = 12.5      # halbe Kantenlaenge in mm
2
3 while not self._stop_tracking_flag:
4     pos = self._get_cube_world_position()
5     if pos is not None:
6         cx, cy, cz = pos[0]*1000, pos[1]*1000, pos[2]*1000
7
8         node = self._ros_node
9         if node:
10            r = node.last_pose.get('r', 0.0)
11            rx = node.last_pose.get('x', cx)
12            ry = node.last_pose.get('y', cy)
13            rz = node.last_pose.get('z', cz)
14        else:
15            r = HOME_R
16            rx, ry, rz = HOME_X, HOME_Y, HOME_Z
17
18        dx, dy, dz = rx - cx, ry - cy, rz - cz
19        dist = math.sqrt(dx*dx + dy*dy + dz*dz)

```

```

20         if dist > 1.0:
21             nx, ny, nz = dx/dist, dy/dist, dz/dist
22             tx = cx + nx * CUBE_HALF_MM
23             ty = cy + ny * CUBE_HALF_MM
24             tz = cz + nz * CUBE_HALF_MM
25         else:
26             tx, ty, tz = cx, cy, cz
27
28         device = node.device if node else None
29         move_fn = getattr(device, 'move_to', None) if device else None
30         if callable(move_fn):
31             move_fn(tx, ty, tz, r)
32
33     _time.sleep(1.0 / max(0.1, self._tracking_rate_hz))

```

Listing 13: Tracking-Thread-Kernlogik

Die Weltposition wird via `UsdGeom.Xformable.ComputeLocalToWorldTransform()` exakt ausgelesen (inkl. Elternttransformationen). Tracking-Frequenz: 0,5–10,0 Hz, einstellbar per Slider.

9.3 Betrieb Schritt für Schritt

Normalbetrieb (Digital Twin):

1. Szene mit Dobot-Magician-Modell laden.
2. ArticulationRootAPI auf `base_link` prüfen (*Property* → *Physics* → *Articulation Root*).
3. COM-Port eingeben (Standard: COM3) und **Connect** klicken. Status: *Verbunden – warte auf Play...*
4. **Play** drücken. Status: *Modell verbunden: 4/4 DOFs gefunden.*
5. Simulationsmodell folgt echtem Roboter in Echtzeit.

Pick-&-Place-Workflow (Teaching):

1. Tab *Pick & Place* wählen.
2. Roboter manuell in Ausgangsposition fahren (Jog-Tasten).
3. *Rec Punkt* klicken.
4. An Greifpunkt: *Saugen (Aufn.)* auf EIN – automatisch Waypoint gespeichert.
5. Schritte 2–4 für alle Punkte wiederholen.
6. *Play* starten; Sequenz läuft mit Sicherheits-Anfahrt.
7. *Stop* bricht jederzeit ab.

10 ROS2-Integration

Die ROS2-Anbindung ermöglicht externen Knoten, die Gelenkzustände des Dobot in Echtzeit zu empfangen.

10.1 Node-Konfiguration

Parameter	Wert
Node-Name	dobot_extension_bridge
Topic	/joint_states
Nachrichtentyp	sensor_msgs/msg/JointState
Queue-Tiefe	10
Publish-Frequenz	≈ 60 Hz (Frame-Rate-gekoppelt)
RMW	rmw_fastrtps_cpp (Fast DDS)
ROS2-Distro	Humble (eingebettet in Isaac-Sim-Bridge)

10.2 Nachrichtenstruktur

```

1 msg.header.stamp = self._node.get_clock().now().to_msg()
2 msg.name         = ['joint_1', 'joint_2', 'joint_5', 'joint_6']
3 msg.position     = [math.radians(j1),          # Radiant (ROS2-Konvention)
4                     math.radians(j2),
5                     math.radians(pos_j5),
6                     math.radians(pos_j6)]
7 # velocity und effort bleiben leer (optionale Felder)

```

Listing 14: JointState-Nachricht in update_step()

10.3 ROS2-Kontext-Verwaltung

`rclpy.init()` wird genau einmal aufgerufen (`rclpy.ok()`-Guard verhindert Mehrfach-Init). `rclpy.spin_once(timeout_sec=0.0)` am Ende von `update_step()` verarbeitet eingehende Nachrichten nicht-blockierend.

11 Konstanten (constants.py)

Alle magischen Zahlen und Konfigurationswerte sind in `constants.py` zentralisiert, damit sie an einer Stelle gepflegt werden können.

11.1 Farbwerte (ARGB-Hex)

Konstante	Hexwert	Verwendung
CLR_BG_DARK	0xFF0A0F1A	Sehr dunkler Hintergrund
CLR_BG_MID	0xFF101828	Aktives Panel / Toolbar
CLR_ACCENT	0xFF4A9EFF	Info / Akzentfarbe
CLR_GREEN	0xFF4ADE80	Erfolg / Verbunden
CLR_RED	0xFFEF6B6B	Fehler / Getrennt
CLR_YELLOW	0xFFE0B040	Warnung / Pending
CLR_TEXT	0xFFD0D8E8	Primärtext
CLR_TEXT_DIM	0xFF607090	Sekundärtext
CLR_BORDER	0xFF1A2840	Trennlinien

11.2 Nullposition und Servo

Konstante	Wert	Beschreibung
HOME_X	−3,7 mm	Nullposition X
HOME_Y	227,8 mm	Nullposition Y
HOME_Z	66,1 mm	Nullposition Z
HOME_R	88,4°	Nullposition Wristdrehung
GP3_PWM_ADDRESS	15	IO-Adresse PWM1 am GP3-Port
SERVO_FREQ_HZ	50,0 Hz	RC-Servo-Trägerfrequenz
SERVO_MIN_DUTY	0,05	1 ms-Puls = 0°
SERVO_MAX_DUTY	0,10	2 ms-Puls = 180°

11.3 USD-Pfade

Konstante	Pfad
ROBOT_USD_PATH	/World/magician/base_link

12 Bekannte Einschränkungen

- **ROS2 eingebettet:** Die Extension verwendet die in Isaac Sim eingebettete ROS2 Humble Bridge. Eine separate ROS2-Installation ist nicht erforderlich, jedoch auch nicht kompatibel ohne Anpassung der Bibliothekspfade.
- **DC-API vor Play:** `get_articulation()` vor dem ersten Play-Ereignis erzeugt eine Warnung. Durch Lazy-Init tritt diese nur einmal auf.
- **Suction-Methoden-Varianz:** Die Fallback-Liste deckt bekannte pydobot-Versionen ab; bei zukünftigen Versionen ggf. erweitern.
- **Servo-Zustand nach Reconnect:** Nach Disconnect/Reconnect startet der Servo-Winkel bei 90° – der tatsächliche Hardware-Winkel kann abweichen.
- **Keine Greifer-Rückmeldung:** Saugbefehle werden als Fire-and-Forget gesendet. Ob ein Objekt tatsächlich gegriffen ist, kann ohne Drucksensor nicht verifiziert werden.
- **Physikalisch ungültige Winkel:** Winkel außerhalb $[-2\pi, 2\pi]$ führen zum PhysX-Fehler in `set_dof_position_target()`. Die Kinematik-Formeln halten alle Ausgaben im gültigen Bereich.